

## Partie 4 : Gestion des Exceptions et Remaniement

### 2.1 Indication des cas d'erreur (Tests d'exceptions)

#### Pourquoi une seule exception est levée et non les deux ?

L'exécution d'un test s'arrête dès qu'une exception est levée. Si la première instantiation est invalide, elle interrompt la méthode immédiatement ; la seconde ligne n'est donc jamais atteinte ni évaluée.

#### Pourquoi faut-il deux blocs try/catch ?

Pour garantir l'**indépendance des tests**. Si l'on veut vérifier que deux cas d'erreurs distincts déclenchent bien des exceptions, les placer dans le même bloc ferait que le succès du premier empêcherait l'exécution du second. Deux blocs permettent de valider chaque scénario séparément.

#### Que se passe-t-il si aucune exception n'est levée ?

Le programme continue son exécution linéaire et atteint l'instruction `fail()`. Cela force l'échec du test JUnit, indiquant que le code n'a pas réagi (n'a pas levé d'exception) alors qu'il aurait dû le faire face à une entrée invalide.

#### Quelle est la valeur de `inverse` et `egale` dans le catch ?

Elles valent `null`. Comme l'exception est levée durant l'appel au constructeur (avant que l'objet ne soit créé et retourné), l'affectation à la variable n'a jamais lieu.

#### Comment utiliser les variables dans try et catch ?

Pour être accessibles à la fois dans le bloc `try` et dans le bloc `catch` (ou après), les variables doivent être **déclarées avant** le bloc `try`.

---

### 1.3 Classe de base de l'exception `java.lang.RuntimeException`

#### Compiler et exécuter avec `NullPointerException`. Que se passe-t-il ?

- **Résultat** : La compilation réussit, l'exécution lève l'exception et le test échoue.
- **Explication** : `NullPointerException` est une exception **non contrôlée** (*unchecked*). Le compilateur n'impose pas de bloc `try/catch` ni de clause `throws`.

#### Remplacer `NullPointerException` par `ClassNotFoundException`. Que se passe-t-il ?

- **Résultat** : Erreur de compilation si l'exception n'est pas gérée.

- **Explication** : `ClassNotFoundException` est une exception **contrôlée** (*checked*). Le Java impose que cette exception soit soit capturée, soit propagée.

### Pourquoi cette exception est contrôlée et pas l'autre ?

Tout dépend de la hiérarchie de classes :

- `NullPointerException` hérite de `RuntimeException` → **Non contrôlée** (erreurs de programmation fréquentes).
- `ClassNotFoundException` hérite de `Exception` (mais pas de `RuntimeException`) → **Contrôlée** (événements extérieurs prévisibles).

### Comment corriger les autres erreurs sans capturer l'exception ?

En ajoutant la clause `throws ClassNotFoundException` à la signature (le prototype) de la méthode de test.

### Pourquoi la clause `throws` est-elle nécessaire ?

Parce que le compilateur doit être informé que la méthode est susceptible de propager une exception contrôlée vers l'appelant, afin que celui-ci puisse la gérer à son tour.

---

## 2.2 Masquage d'information du packaging (Encapsulation)

### 1. Quel est le problème de portée ?

Si les classes concrètes (ex: `JaugeNaturel`) sont `public`, n'importe quel développeur peut faire un `new JaugeNaturel(...)`. Cela crée un couplage fort avec l'implémentation. Le client devrait dépendre uniquement de l'interface `IJauge`.

### 2. Solution ?

Passer les classes concrètes en visibilité "**package-private**" (enlever le mot-clé `public`). On utilise ensuite une **Factory** publique qui, elle, est autorisée à instancier les classes du packaging et à les retourner sous forme d'interface.

### 3. Désavantage ?

Cela ajoute une couche d'abstraction supplémentaire (la Factory), ce qui peut rendre le code légèrement plus complexe à naviguer pour un développeur débutant.

---

## 2.3 Un peu de remaniement de code

Dans quel cas faudrait-il tester ce nouveau code ?

Il est crucial de tester :

1. **L'état initial** : Vérifier que les attributs (nom, destination) sont correctement affectés après l'instanciation.
  2. **La robustesse** : Vérifier qu'une destination invalide (négative) lève bien l'exception prévue.
  3. **La logique métier** : Vérifier que le formatage du nom (ex: "PassagerStandard5") respecte strictement la règle de construction.
- 

## 2.4 Fin de l'itération

### 1. Expliquer le problème de duplication de code qui reste dans le code

Le code `passagers.add(p)` est répété dans plusieurs méthodes. Cette duplication nuit à la maintenance : si la logique d'ajout change (ex: vérification de doublons), il faudra modifier le code à plusieurs endroits.

**Expliquer le problème de duplication d'instance** On peut créer deux objets différents en mémoire qui représentent la même réalité physique (même nom, même destination).

Java

```
PassagerStandard p1 = new PassagerStandard("Dupont", 5);  
PassagerStandard p2 = new PassagerStandard("Dupont", 5);
```

2. Sans mécanisme de contrôle (comme un `Cache` ou la redéfinition de `equals()`), le système traitera `p1` et `p2` comme deux personnes distinctes, ce qui est une erreur logique.
3. **Avez-vous noté d'autres problèmes ?**
  - **Validation des capacités** : L'absence de contrôle sur les paramètres de l'Autobus (on peut créer un bus avec -10 places).
  - **Cohérence du trajet** : Un passager peut demander un arrêt qui n'existe pas sur la ligne du bus.